Practice 4 — Half-Term Challenge: Grabbing an Object

Robotics and Cyber-Physical Systems
School of Sciences and Engineering
Tecnológico de Monterrey

Introduction

This is the half-term challenge. You will build a complete autonomous manipulation pipeline that combines everything covered so far: forward kinematics, 3D object localization, visual servoing, and multi-phase state machine control.

The goal is simple to describe: *find a blue object, drive toward it, position the gripper, and pick it up*. But achieving this reliably requires coordinating multiple control loops, switching between cameras, and managing robot state safely.

How This Practice Works

Rather than writing one large script, you build five progressively more capable scripts:

Script	New capability added	Phases active
`p4_1_locate_object.py`	3D localization and plotting	— (stationary)
`p4_2_approach.py`	Autonomous approach	approach
`p4_3_align.py`	Fine alignment	approach → align
`p4_4_reach.py`	Arm extension	approach → align → reach
`p4_5_grab.py`	Gripper close	approach → align → reach → grab

Each script is a direct copy of the previous one with one new block of logic added. **Always verify each script works before building on it**—a bug in Part 2 will be much harder to diagnose after Part 4 layers code on top.

Before You Begin

Update your repository and synchronize dependencies:

```
git pull
git fetch origin
git reset --hard origin/main
uv sync
```

Simulator Starting Position

The file `stretch_toolkit/sim_config.json` controls where the robot spawns in the simulator. If you have used this repository before, you may recognise it as the renamed `robocasa_config.json`—it serves the same purpose but now includes additional settings such as the starting position.

```
{
  "start_translation": [0, 1.0, 0]
}
```

The values are `[x, y, z]` in metres. The blue object (on the table) is at $y \approx -1.0$, so increasing `y` moves the robot further away. The default `y = 1.0` places the robot about 2 m from the table—a good starting distance for the approach phase. If the robot spawns too close or too far, adjust this value before running your scripts.

Reference: key tools in `stretch_toolkit`

- `RobotTransforms(controller)` — runs forward kinematics on the current joint state to compute where each link and camera is in the world. Must be constructed before the loop.
- `locate_object(centroid, depth_cam, transforms)` — given a pixel centroid from a depth camera, returns `(obj2cam_T, obj2base_T)`: the object's 4×4 homogeneous pose in camera frame and robot base frame. The position in meters is the last column: `T[0:3, 3]`.
- `ObjectPlotter()` — opens a live 3D matplotlib window. Call `plotter.update(cam_T, obj_T)` each loop tick. Pass `None` for `obj_T` when no object is detected. Call `plotter.close()` in the `finally` block.
- `StateController(controller, targets)` — a proportional controller that drives a set of joints to target positions. `get_command()` returns velocity commands each tick. `is_at_goal()` returns `True` once all joints are within tolerance.
- `merge_proportional(primary, secondary)` — blends two velocity dictionaries. The `primary` (manual teleop) takes priority when non-zero; `secondary` (autonomous) fills in otherwise.
- `HEAD_CAMERA`, `WRIST_CAMERA` — depth camera info objects (used by `locate_object` and `.get_depth()`).
- `HEAD_RGB_CAMERA`, `WRIST_RGB_CAMERA` — RGB camera objects. Call `.get_frame()` to get the current image as a NumPy array. Always check for `None` before use.

The Main Loop Pattern

Every script in this practice follows the same main loop structure. The pattern below is your template—write all your per-phase logic inside the loop:

```
while True:
    velocities = teleop.get_normalized_velocities()  # manual override input
    auto_velocities = {}                              # autonomous commands go here

    # --- your logic: populate auto_velocities based on sensor readings ---

    velocities = merge_proportional(velocities, auto_velocities)
    controller.set_velocities(velocities)
    cv2.waitKey(1)
    time.sleep(1 / 30)  # 30 Hz
```

Note

`merge_proportional` is called *after* your logic populates `auto_velocities`. Do not call `controller.set_velocities` inside individual phase branches—call it once at the bottom of the loop.

Part 1 — Object Localization and 3D Visualization

Objective

Detect a blue object using HSV color segmentation, compute its 3D position in the robot's base frame, and visualize it in a live 3D plot. This is the perception foundation the later phases depend on.

Task

Create `p4_1_locate_object.py` from the boilerplate with these imports:

```
from stretch_toolkit import (
    controller, teleop, merge_proportional, locate_object,
    BACKEND_NAME, HEAD_CAMERA, HEAD_RGB_CAMERA,
    RobotTransforms, ObjectPlotter
)
import time
import cv2
import numpy as np

print(f"\n=== Running on {BACKEND_NAME} backend ===\n")
```

Step 1: Write the Object Detection Function

Add this function *above* `main()`. It converts the image to HSV, creates a binary mask for blue pixels, finds contours, and returns the centroid of the largest one:

```
def find_blue_object(rgb_frame):
    if rgb_frame is None:
        return None
    hsv = cv2.cvtColor(rgb_frame, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv,
                       np.array([110, 100, 100]),
                       np.array([130, 255, 255]))
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)
    if contours:
        largest = max(contours, key=cv2.contourArea)
        M = cv2.moments(largest)
        if M['m00'] > 0:
            cx = int(M['m10'] / M['m00'])
            cy = int(M['m01'] / M['m00'])
            return (cx, cy)
    return None
```

Step 2: Set Up Transforms and Plotter

Inside `main()`, *before* the loop:

```
transforms = RobotTransforms(controller)
plotter = ObjectPlotter()
```

Step 3: Localize the Object Each Tick

Inside the loop, after getting teleop velocities:

```python
rgb = HEAD_RGB_CAMERA.get_frame()
centroid = find_blue_object(rgb)

cam_T = transforms.get_cam_T(HEAD_CAMERA)

if centroid is not None:
    _, obj_T = locate_object(centroid, HEAD_CAMERA, transforms)
else:
    obj_T = None
```

Step 4: Annotate the Image and Update the Plotter

Draw the detected centroid and the measured depth on the RGB image:

```python
if centroid is not None and rgb is not None:
    depth = HEAD_CAMERA.get_depth(centroid)
    cv2.circle(rgb, centroid, 8, (0, 255, 0), -1)
    if depth is not None:
        cv2.putText(rgb, f"{depth:.2f}m",
                    (centroid[0] + 10, centroid[1]),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

plotter.update(cam_T, obj_T)

if rgb is not None:
    cv2.imshow("Head RGB", rgb)
```

Step 5: Clean Up on Exit

In the `finally` block, call `plotter.close()` in addition to the standard cleanup:

```python
finally:
    controller.set_velocities({})
    controller.stop()
    cv2.destroyAllWindows()
    plotter.close()
    print("Done.")
```

Expected Result

Run the script and pan the head toward a blue object on a surface:

```
uv run p4_1_locate_object.py
```

You should see:

- **Head RGB window**: the camera view with a green dot on the blue object and a depth label in meters.
- **3D Plotter window**: a blue cone (camera) and a red dot (object). As you tilt and pan the head, the cone moves but the red dot stays fixed—this confirms the transform is working correctly.

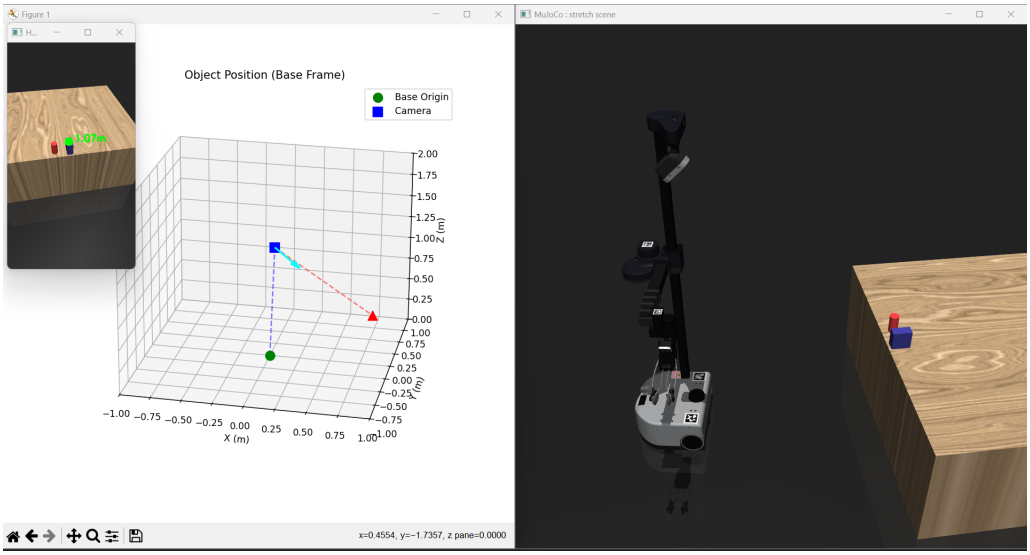


Figure 1: Left: Head RGB camera with green centroid dot and depth label. Right: 3D plotter showing camera frustum (blue cone) and object position (red dot) in the robot base frame.

Before Moving On

Green dot appears on the blue object in the RGB window

Depth label shows a reasonable distance in metres (e.g. 0.8m–2.0m)

Red dot in the 3D plotter stays approximately fixed when you pan/tilt the head

Script exits cleanly with Ctrl+C and both windows close

Common Mistake

If the depth label shows `None` or the red dot jumps erratically, the object may be at the edge of the depth camera's range, or the object is too small in the frame. Move the object closer (within 1.5 m) or use a larger object.

Part 2 — Autonomous Approach

Objective

Drive toward the object and stop at ∼50 cm with the object to the robot's right (the "flank" position). The wrist camera faces sideways, so presenting the right flank gives it a clear view of the object for the next two phases.

Background: Base-Frame Coordinates

Extract the object position from `obj2base_T[0:3, 3]` as `x, y, z`. In the robot base frame: $+x$ is ahead, $+y$ is left, $+z$ is up. The horizontal angle to the object is $\theta = \text{atan2}(y, x)$, where $\theta = 0$ is straight ahead and $\theta = -\pi/2$ is directly to the right.

Task

Copy `p4_1_locate_object.py` to `p4_2_approach.py`. Remove the `ObjectPlotter` import and construction—you no longer need the 3D visualization. Add `StateController` and `math` to your imports.

Declare all tuning constants at the top of `main()`, before the loop:

```
Kp_pan     = 1.0
Kp_tilt    = 1.0
Kp_angle   = 5.0 / math.pi
Kp_forward = 2.0
Kp_lift    = 5.0
```

Also declare the hysteresis state variable before the loop:

```
in_zone = False
```

Step 1: Stow Pose — Keep the Arm Safe While Driving

Create a `StateController` that holds the arm retracted and the gripper in a safe position while the robot drives. If the arm is extended during base motion, it could collide with the environment:

```
stow_pose = StateController(controller, {
    "wrist_roll_counterclockwise": 0.0,
    "wrist_yaw_counterclockwise": 0.0,
    "wrist_pitch_up": 0.0,
    "gripper_open": 0.3,
    "arm_out": 0.0,
})
```

At the *end* of the loop (after your logic), merge this into the velocities:

```
velocities = merge_proportional(velocities, stow_pose.get_command())
```

Step 2: Head Tracking — Keep the Object in View

Inside the centroid-found block, servo the head to keep the object centered in the frame. Compute normalized pixel errors (in the range −0.5 to +0.5) and apply proportional control:

```
cx, cy = centroid
frame_cx = rgb.shape[1] / 2
frame_cy = rgb.shape[0] / 2
```

```
error_x = (cx - frame_cx) / rgb.shape[1]
error_y = (cy - frame_cy) / rgb.shape[0]
auto_velocities["head_pan_counterclockwise"] = -Kp_pan * error_x
auto_velocities["head_tilt_up"]              = -Kp_tilt * error_y
```

Step 3: Localize and Compute Angle/Distance

Directly after head tracking, call `locate_object` with the head camera and extract position:

```
_, obj2base_T = locate_object((cx, cy), HEAD_CAMERA, transforms)
if obj2base_T is not None:
    x, y, z = obj2base_T[0:3, 3]
    angle_z             = math.atan2(y, x)
    horizontal_distance = math.sqrt(x**2 + y**2)
```

All of the base motion logic (Steps 4–6 below) goes inside this `if obj2base_T is not None:` block.

Step 4: Base Rotation and Gated Forward Motion

Rotate the base toward the object and drive forward—but only when well-aligned (within ∼18°) to avoid driving sideways:

```
auto_velocities["base_counterclockwise"] = -Kp_angle * angle_z
```

Moving forward when badly misaligned is dangerous—the robot might drive sideways. Gate forward motion on alignment by computing how well-aligned the robot is and only allowing forward speed in the last 10% of perfect alignment:

```
alignment   = 1.0 - abs(angle_z) / math.pi   # 1.0 = facing it, 0.0 =
    180 deg away
travel_auth = max(0.0, min(1.0, (alignment - 0.9) / 0.1))
auto_velocities["base_forward"] = Kp_forward * horizontal_distance *
    travel_auth
```

Note

`travel_auth` is 0 unless `alignment > 0.9`, meaning the robot must be within about 18° of facing the object before it will drive forward. This prevents the robot from driving sideways while turning.

Step 5: Flank Presentation with Hysteresis

When within 45–55 cm, stop advancing and rotate to the right flank. The `in_zone` flag prevents oscillation: the robot must drift outside 40–60 cm to exit flank mode:

```
# Update zone membership with hysteresis
if not in_zone:
    if 0.45 <= horizontal_distance <= 0.55:
        in_zone = True
else:
    if horizontal_distance < 0.4 or horizontal_distance > 0.6:
        in_zone = False

# Choose behavior based on zone
if in_zone:
```

```
    # Rotate to flank angle (-pi/2 = object to the right)
    angle_error = -math.pi / 2 - angle_z
    auto_velocities["base_counterclockwise"] = Kp_angle * angle_error
    auto_velocities["base_forward"] = 0.0
    print(f"\rDist: {horizontal_distance:.2f}m  "
          f"Angle Err: {math.degrees(angle_error):+6.1f}deg  [FLANK]  ",
          end="", flush=True)
else:
    # Face and drive toward object (code from Step 4)
    auto_velocities["base_counterclockwise"] = -Kp_angle * angle_z
    alignment   = 1.0 - abs(angle_z) / math.pi
    travel_auth = max(0.0, min(1.0, (alignment - 0.9) / 0.1))
    auto_velocities["base_forward"] = Kp_forward * horizontal_distance
        * travel_auth
    print(f"\rDist: {horizontal_distance:.2f}m  "
          f"Align: {alignment:.3f}  Auth: {travel_auth:.2f}  [moving]  ",
          end="", flush=True)
```

Step 6: Lift Pre-positioning

Start raising the lift during approach so it's already at the right height when flanked. The `+0.10` offset targets 10 cm above the object, leaving room to descend during alignment:

```
cam_z = transforms.get_wrist_cam_T()[2, 3]
auto_velocities["lift_up"] = Kp_lift * (z - (cam_z + 0.01) + 0.10)
```

Expected Result

Place a blue object at table height in front of the robot and run:

```
uv run p4_2_approach.py
```

Watch the terminal output. You should see the `[moving]` state as the robot drives toward the object, then the `[FLANK]` state as it rotates. The robot should come to rest with the object approximately to its right.

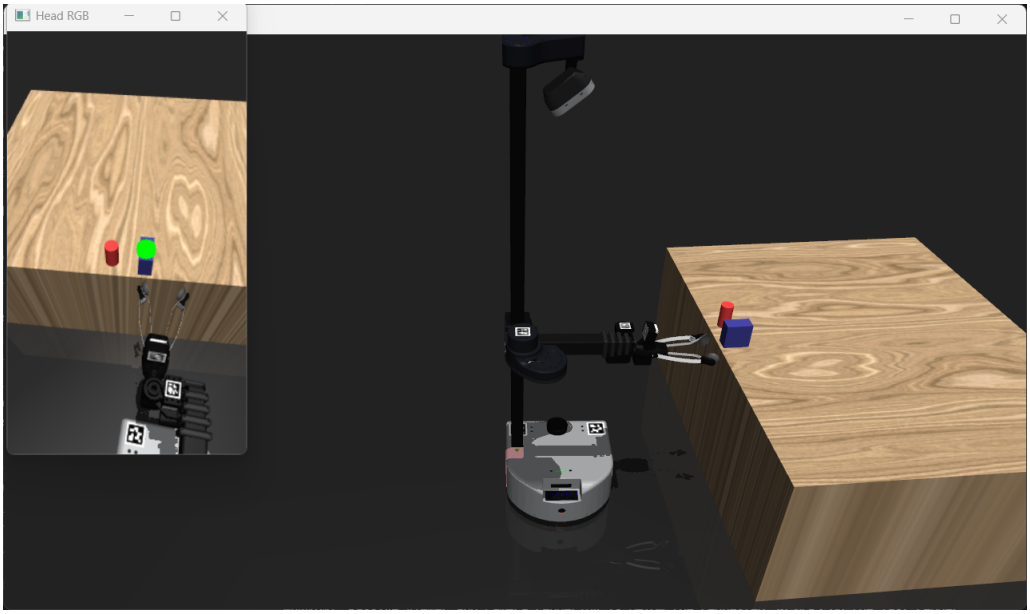


Figure 2: Robot in flank position with the blue object to its right. Terminal shows `[FLANK]` with a small angle error.

Before Moving On

Terminal shows `[moving]` while the robot drives toward the object

Terminal switches to `[FLANK]` when the robot is within 45–55 cm

The angle error printed in `[FLANK]` mode approaches 0 as the robot rotates

The lift rises during approach (the arm moves upward)

The arm stays retracted (`arm_out = 0`) throughout

Common Mistake

If the robot drives past the object without entering flank mode, the object may be too high or far away for the depth reading to be reliable. Try placing the object within 1.5 m and ensuring it is clearly visible from the starting position.

Part 3 — Fine Alignment

Objective

The approach phase gets the robot roughly into position, but small angle and height errors remain. This phase switches to the wrist camera—mounted near the gripper—to fine-tune base rotation and lift height until the gripper is precisely lined up.

Task

Copy `p4_2_approach.py` to `p4_3_align.py`. Add `WRIST_CAMERA` and `WRIST_RGB_CAMERA` to your imports from `stretch_toolkit`.

Step 1: Add Phase State Variable

Before the loop (alongside `in_zone`), add:

```
phase = "approach"
print(f"Phase: {phase}")
```

Step 2: Wrap Approach Logic in a Phase Check

Wrap your existing approach block in `if phase == "approach":`. Nothing inside it changes—you are just gating it:

```
if phase == "approach":
    # ... everything from Part 2 ...
```

Step 3: Add the Approach-to-Align Transition

Inside the `in_zone` branch of the approach logic, after the print statement, add a transition condition. When the flank angle error is small enough, the robot is correctly oriented—switch to the align phase and close the head camera window:

```
if in_zone:
    angle_error = -math.pi / 2 - angle_z
    auto_velocities["base_counterclockwise"] = Kp_angle * angle_error
    auto_velocities["base_forward"] = 0.0
    print(f"\rDist: {horizontal_distance:.2f}m  "
          f"Angle Err: {math.degrees(angle_error):+6.1f}deg  [FLANK]
              ",
          end="", flush=True)
    # Transition to align once roughly flanked
    if abs(angle_error) < math.radians(5):
        cv2.destroyWindow("Head RGB")
        phase = "align"
        print(f"\nPhase: {phase}")
```

Step 4: Add the Align Phase

After the approach block, add:

```
elif phase == "align":
    rgb = WRIST_RGB_CAMERA.get_frame()
    centroid = find_blue_object(rgb)

```

```
    if centroid is not None:
        cx, cy = centroid
        _, obj2base_T = locate_object((cx, cy), WRIST_CAMERA,
            transforms)
        if obj2base_T is not None:
            x, y, z = obj2base_T[0:3, 3]
            angle_z = math.atan2(y, x)
            cam_z   = transforms.get_wrist_cam_T()[2, 3]

            # Rotate base toward final flank target (3 deg bias for
                gripper offset)
            auto_velocities["base_counterclockwise"] = \
                Kp_angle * (-math.pi / 2 - angle_z + math.radians(3))
            # Move lift until object is level with wrist camera
            auto_velocities["lift_up"] = Kp_lift * (z - (cam_z + 0.01))

            # Check transition conditions
            angle_err = abs(-math.pi / 2 - angle_z)
            lift_err  = abs(z - (cam_z + 0.01))
            if stow_pose.is_at_goal() and \
               angle_err < math.radians(5) and \
               lift_err < 0.03:
                phase = "done"
                print(f"\nPhase: {phase}")

        if rgb is not None:
            cv2.circle(rgb, (cx, cy), 10, (0, 255, 0), -1)

    if rgb is not None:
        cv2.imshow("Wrist RGB", rgb)

    velocities = merge_proportional(velocities, auto_velocities)
    velocities = merge_proportional(velocities, stow_pose.get_command()
        )
```

Note

The 3° bias (`math.radians(3)`) in the base rotation command compensates for the physical offset between the wrist camera lens and the gripper fingertips. Without it, the camera would be centered on the object but the gripper would be slightly off to one side.

Step 5: Add the Done Phase

The `"done"` phase simply holds the robot in place (no commands). Add it after the align block:

```
elif phase == "done":
    pass  # Hold position
```

Expected Result

```
uv run p4_3_align.py
```

The approach phase runs as before. When the flank angle is achieved, the `Head RGB` window closes, the `Wrist RGB` window opens, and the terminal prints `Phase: align`. The robot makes small adjustments to rotation and lift. When all three conditions are met, it prints `Phase: done` and holds.

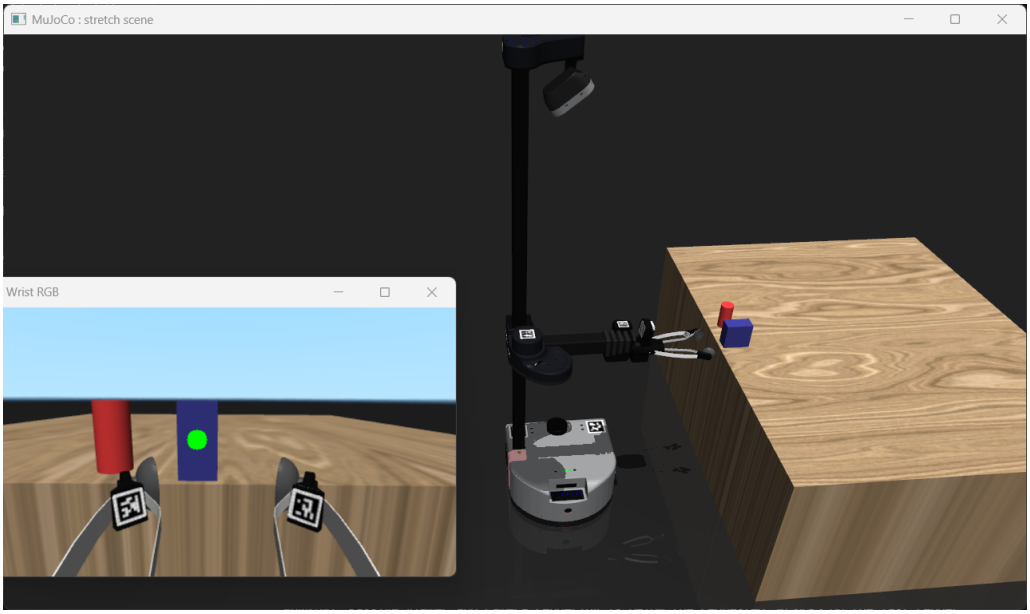


Figure 3: Wrist RGB camera view during the align phase. The blue object's centroid (green dot) is approximately centered in the frame, indicating the robot is ready to reach.

Before Moving On

Terminal prints `Phase: align` after the head window closes

The wrist camera window opens and shows the blue object

The robot makes small adjustments (not large oscillations)

Terminal eventually prints `Phase: done` and the robot stops

Common Mistake

If the align phase never transitions to `"done"`, the most common cause is that `stow_pose.is_at_goal()` is never `True`. Verify that you are calling `merge_proportional(velocities, stow_pose.get_command())` at the end of the align block—if the stow pose commands are not being sent, the joints never reach their targets.

Part 4 — Arm Extension and Wrist Servoing

Objective

Extend the arm while using the wrist camera to keep the object centered. The arm stops at 12 cm depth—the point where the gripper fingers reach the object.

Task

Copy `p4_3_align.py` to `p4_4_reach.py`.

Step 1: Add Tuning Constants

Add these constants alongside the others at the top of `main()`:

```
Kp_yaw   = 0.5
Kp_pitch = 0.5
Kp_arm   = 10.0
```

Step 2: Add the Pre-Grip Pose Controller

Add a second `StateController` for use during the reach phase. This opens the gripper slightly and straightens the wrist roll, so the gripper arrives at a good orientation:

```
pre_grip_pose = StateController(controller, {
    "wrist_roll_counterclockwise": 0.0,
    "gripper_open": 0.4,
})
```

Step 3: Change the Align Transition Target

In the align block, change:

```
phase = "done"
```

to:

```
phase = "reach"
```

Step 4: Add the Reach Phase

After the align block and before the done block, add:

```
elif phase == "reach":
    rgb = WRIST_RGB_CAMERA.get_frame()
    centroid = find_blue_object(rgb)

    if centroid is not None:
        cx, cy = centroid

        # Servo wrist to keep object centered
        if rgb is not None:
            frame_cx = rgb.shape[1] / 2
            frame_cy = rgb.shape[0] / 2
            error_x = (cx - frame_cx) / rgb.shape[1]
            error_y = (cy - frame_cy) / rgb.shape[0]
            auto_velocities["wrist_yaw_counterclockwise"] = Kp_yaw *
                error_x
```

```
            auto_velocities["wrist_pitch_up"]              = -Kp_pitch *
                error_y
            cv2.circle(rgb, (cx, cy), 10, (0, 255, 0), -1)

        # Extend arm based on depth
        distance = WRIST_CAMERA.get_depth((cx, cy))
        if distance is not None:
            distance_error = distance - 0.12
            auto_velocities["arm_out"] = Kp_arm * distance_error
            print(f"\rDepth: {distance:.3f}m  Error: {distance_error
                :+.3f}m   ",
                  end="", flush=True)
            if abs(distance_error) < 0.02 and pre_grip_pose.is_at_goal
                ():
                phase = "done"
                print(f"\nPhase: {phase}")

    if rgb is not None:
        cv2.imshow("Wrist RGB", rgb)

    velocities = merge_proportional(velocities, auto_velocities)
    velocities = merge_proportional(velocities, pre_grip_pose.
        get_command())
```

Note

Notice that the reach phase uses `pre_grip_pose.get_command()` instead of `stow_pose.get_command()`. This is critical: `stow_pose` would command `arm_out = 0` which would fight against the arm extension. Switch to `pre_grip_pose` which does not constrain the arm.

Expected Result

```
uv run p4_4_reach.py
```

After alignment, the arm should extend toward the object. The terminal will print live depth readings. When the object is 12 cm away and the gripper pose is stable, the script prints `Phase: done` and holds.

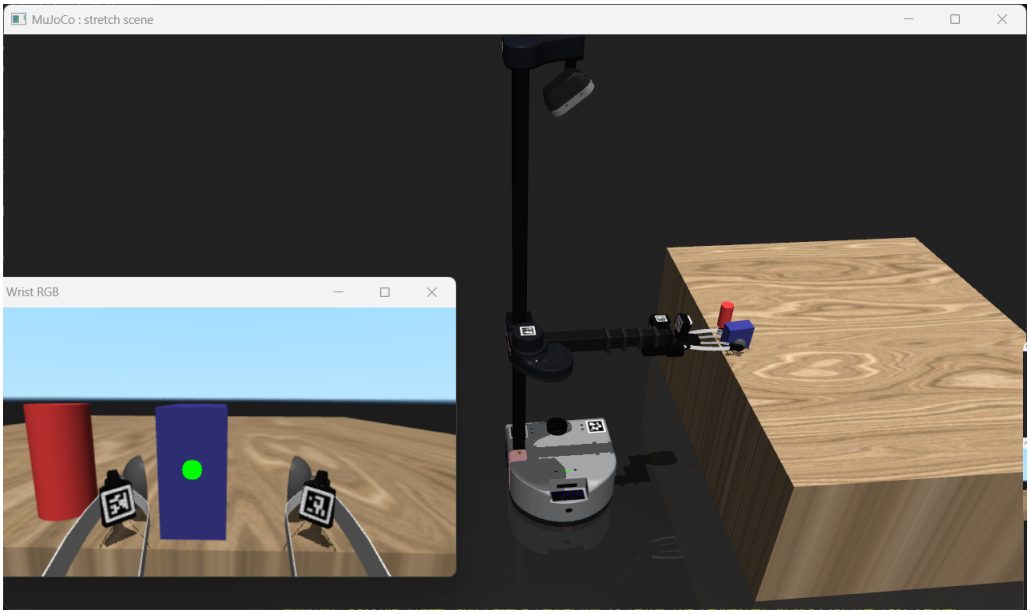


Figure 4: Wrist camera view with the arm extended. The blue object fills more of the frame as the arm approaches. Terminal shows depth decreasing toward 0.12 m.

Before Moving On

Arm extends after alignment (not before)

Terminal shows decreasing depth values as the arm extends

Depth stabilizes near 0.12 m (12 cm)

Terminal prints `Phase: done` when depth is within 2 cm of 0.12 m

Common Mistake

If the arm oscillates (extends and retracts repeatedly without settling), the gain `Kp_arm = 10.0` may be too high. Reduce it to `5.0` if this occurs. If the arm extends but depth never reaches 0.12 m, the object may be out of the arm's reach—try repositioning it closer to the robot.

Part 5 — Grab

Objective

Close the gripper and lift the arm to complete the pick.

Task

Copy `p4_4_reach.py` to `p4_5_grab.py`.

Step 1: Change the Reach Transition Target

In the reach block, change:

```
phase = "done"
```

to:

```
phase = "grab"
```

Step 2: Add the Grab Phase

After the reach block, add:

```
elif phase == "grab":
    auto_velocities["lift_up"]      = 0.2
    auto_velocities["gripper_open"] = -1.0
    velocities = merge_proportional(velocities, auto_velocities)
```

This phase runs indefinitely until you terminate with Ctrl+C.

Note

Do not call `stow_pose.get_command()` or `pre_grip_pose.get_command()` in the grab phase. Both would counteract the gripper close command (`stow_pose` targets `gripper_open = 0.3`; the grab phase needs `gripper_open = -1.0`).

The Complete Pipeline

Your `p4_5_grab.py` implements the full autonomous pick sequence:

1. **Approach** — Head camera tracks object; base drives forward with alignment gating; flank presentation at 50 cm; lift pre-positions 10 cm above target.
2. **Align** — Wrist camera fine-tunes base rotation and lift until all three conditions are met.
3. **Reach** — Arm extends toward object; wrist servos re-center continuously; transition at 12 cm depth.
4. **Grab** — Gripper closes; lift rises.

Expected Result

Place a blue object at table height and run:

```
uv run p4_5_grab.py
```

The terminal should print each phase transition:

```
Phase: approach
Phase: align
Phase: reach
Phase: grab
```

The robot should complete the full sequence and lift the object.

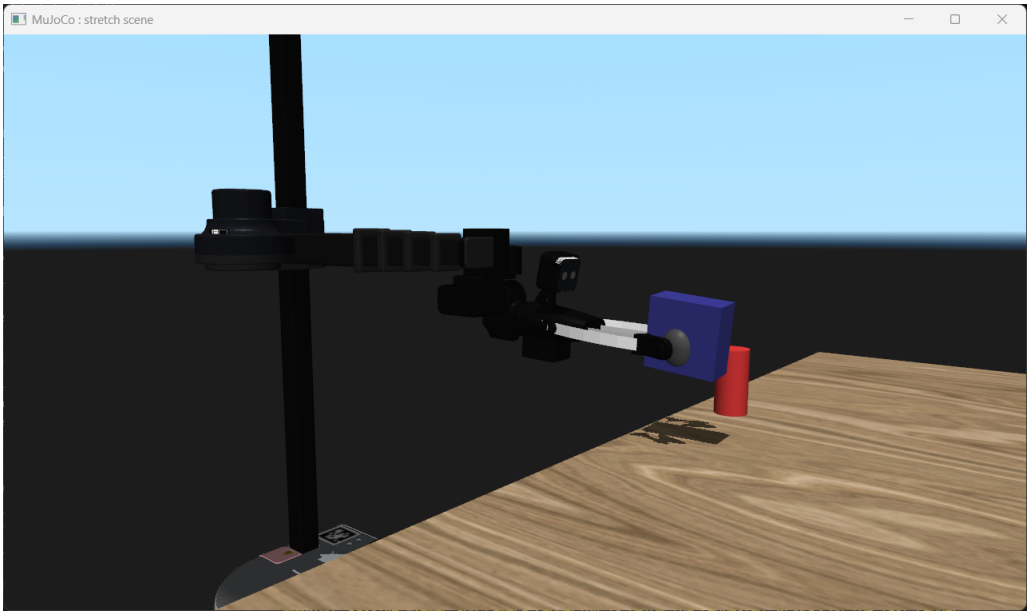


Figure 5: Robot completing the grab: gripper closed around the blue object with the lift raised. Terminal shows all four phase transitions.

Before Moving On

All four phase transitions print in order

Gripper closes in the grab phase

Lift raises after gripper closes

The script exits cleanly with Ctrl+C

Common Mistake

If the gripper closes but doesn't lift the object (it slides through), the align or reach phases may have left the gripper slightly off-center. Run Part 3 or Part 4 in isolation and observe the wrist camera view to check alignment before attempting the full pipeline.

Deliverable

Record a screen capture with voiceover demonstrating your complete system running `p4_5_grab.py` on the simulator. The recording should show the robot completing the full pick without any manual intervention.

Required Content

Your video must include:

1. **All four phase transitions** visible in the terminal output
2. **Approach phase**: the robot driving toward the object, switching to `[FLANK]` mode in the terminal
3. **Align phase**: the wrist camera window opening and the robot making small adjustments
4. **Reach phase**: the arm extending and the depth reading decreasing toward 12 cm in the terminal
5. **Grab phase**: the gripper closing and the lift raising the object

Voiceover

In your voiceover, address the following:

- How does the robot determine where the object is in 3D space? (Mention forward kinematics and the depth camera.)
- Why does the robot present its flank instead of approaching directly?
- Why are there three separate conditions required to transition from align to reach?
- Describe *one change* you would make to the pipeline to make it more robust or reliable.